

Project Design Phase-II

Solution Requirements (Functional & Non-functional)

Date	31 January 2025
Team ID	
Project Name	AI FAQ Assistant API
Maximum Marks	4 Marks

Functional Requirements:

Following are the functional requirements of the proposed solution.

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-1	Account Registration	Registration of an organisation account through a form (email, password, confirm password) Confirmation email sent after registration
FR-2	Login & Access Control	Login with email and password (JWT access and refresh tokens) Token refresh Roles: Admin, Editor, Agent, assigned by an admin
FR-3	API Key Management	Create, view (masked) and revoke API keys for each organisation Each key only reaches its own organisation's data
FR-4	FAQ Management	Create, read, update and delete FAQs Draft, Published and Archived status workflow Categories and tags Search, filter, sort and pagination
FR-5	Bulk Import	Import FAQs from CSV or JSON Report invalid rows with line numbers and reasons
FR-6	Embeddings & Semantic Search	Create or update an embedding whenever an FAQ changes Return the most relevant published FAQs ranked by meaning
FR-7	AI Question Answering	Short answer written only from the matching published FAQs Source FAQ links and a confidence score with every answer Follow-up questions using session history
FR-8	Fallback & Escalation	Clear "I don't know" reply when confidence is below the threshold Escalate a question to a human agent with an email alert Agent queue to view and resolve escalated questions
FR-9	Customer Feedback	Thumbs up or down on each answer
FR-10	Cost & Abuse Control	Rate limiting on question endpoints, configurable per API key Caching of repeated answers in Redis, cleared when FAQs change

FR No.	Functional Requirement (Epic)	Sub Requirement (Story / Sub-Task)
FR-11	Analytics	Top asked questions Unanswered rate and satisfaction score Usage per API key
FR-12	API Documentation	Interactive Swagger (OpenAPI) documentation for every endpoint

Non-functional Requirements:

Following are the non-functional requirements of the proposed solution. Numeric values are targets to be confirmed during testing.

NFR No.	Non-Functional Requirement	Description
NFR-1	Usability	Consistent REST naming and JSON responses. Clear error messages with correct HTTP status codes. Swagger documentation with request and response examples, so a developer can connect a chat widget without extra help. Answers are short, plain-language and show their sources.
NFR-2	Security	Passwords hashed with bcrypt. JWT for staff and hashed API keys for client apps. Role-based guards and strict organisation isolation, so one organisation can never read another's FAQs or conversations. FAQ text and customer questions are treated as data, not instructions, to reduce prompt-injection risk. Input validation, HTTPS only and secrets kept in environment variables, following OWASP Top 10 guidance.
NFR-3	Reliability	Answers must come only from published FAQs, and the API returns a fallback instead of guessing. Timeouts, retries and clear errors when the LLM, embedding or email service fails; FAQ management keeps working. Automated unit and end-to-end tests run in CI before every release.
NFR-4	Performance	Targets: standard (non-AI) endpoints under 300 ms at the 95th percentile; vector search under 200 ms for up to 10,000 FAQs; cached answers under 100 ms; a new AI answer within 10 seconds, with a 30-second timeout. Vector and database indexes and pagination keep queries fast.
NFR-5	Availability	Target: 99.5% monthly uptime. Health-check endpoint, containerised deployment with automatic restart, managed PostgreSQL with daily backups, and graceful degradation: FAQs can still be listed and edited if the AI provider is down.
NFR-6	Scalability	Stateless NestJS instances scale horizontally behind a load balancer. Redis holds shared cache and rate-limit counters. Embedding creation and large imports run through a queue (for example BullMQ). The AI module can later be separated into its own service.